SPViT Controller: Fine-Grained Parallel ViT Splitting & ARIMA Offloading

SPViT (IEEE TMC 2025) • Dynamic Head & Neuron Partitioning for Vision Transformers

(A) What It Does: Fine-Grained Intra-Block Parallel Splitting of Vision Transformers (ViT)

Patch Tokens X (L x D)

1. Head-Width Splitting: Multi-Head Attention (MSA)

Total H Heads split into h_e (Edge) and h_s (Server)

EDGE: Computes h_e Heads in Parallel

Q_e, K_e, V_e -> Softmax(Q K^T / sqrt(d)) V -> O_e
FLOPs_MSA_e = 4 h_e L D^2 + 2 h_e L^2 D

SERVER: Computes h_s = (H - h_e) Heads

Parallel execution + transmitted via TCP socket
FLOPs_MSA_s = 4 h_s L D^2 + 2 h_s L^2 D

2. Neuron-Width Splitting: Feed-Forward (FFN)

Hidden dimension M split into m_e (Edge) and m_s (Server)

EDGE: m_e Neurons (W1_e, W2_e)

Partial Forward: Y_edge = GELU(X W1_e) W2_e
FLOPs_FC_e = L (m_e + 1) D

SERVER: m_s = (M - m_e) Neurons

Partial Forward: Y_server = GELU(X W1_s) W2_s
FLOPs_FC_s = L (m_s + 1) D

Exact Sum: Y = Y_e + Y_s (Lossless)

Key Advantage: True intra-layer concurrency balances Edge and Server execution without pipeline bubbles

(B) How It Works: Real-Time ARIMA Forecasting & Dynamic Load Balancing (AdaptiveSplit)

1. Runtime Profiling

Timestamps & History

t_e : Edge complete time

t_s : Server start time

t_r : Server reply time

Compute Capability:

FLOPS_d = (FLOPs_a + FLOPs_f)
/ (t_e - t_s)

Network Bandwidth:

BW_d = Params / (t_r - t_s)

2. ARIMA Predictor

ARIMA(p=5, d=1, q=0)

Sliding Window Sequences:

seq_flops[d], seq_bw[d]

First-order Differencing:

Delta y_t = y_t - y_{t-1}

Autoregressive Forecast:

pre_flops_d = ARIMA.fit()

pre_bw_d = ARIMA.fit()

Online forecast of next-step
processing speed & bandwidth

3. Projected Latency DL

Delay Model compute_DL()

Device Projected Delay DL_d:

DL_d = (Params / pre_bw_d)
+ (FLOPs_a / pre_flops_d)
+ (FLOPs_f / pre_flops_d)

Imbalance Metric eta:

eta = |max(DL) - min(DL)|
- lambda_tolerance

Status Check:

eta <= 0: Workloads balanced

eta > 0: Rebalance required

4. Greedy Offloading

adaptive_vit_offloading()

If eta > 0 (imbalance):

1. Rebalance Heads:
for h in 1..head_count:
d* = argmin(DL)
Assign head h to d*
DL[d*] += FLOPs_h / FLOPS

2. Rebalance Neurons:
for n in 1..neuron_count:
d* = argmin(DL)
Assign neuron n to d*
DL[d*] += FLOPs_n / FLOPS

Output: Optimal (h_e, m_e)

Continuous Feedback Loop: Dynamically adapts (h_e, m_e) per minibatch under varying network & edge load